

1 AVL-деревья

1.1 Домашнее задание

1. Покажите, что удаление из AVL-дерева требует $\Omega(\log n)$ вращений в худшем случае.
2. Придумайте, как реализовать структуру данных, поддерживающую следующие операции на последовательности из чисел:
 - `insert(i, x)` — добавление числа x на позицию i (если позиция была занята, то число, которое на ней было, и все числа правее сдвигаются вправо на 1),
 - `twiddle(l, r)` — обмен местами последовательных пар соседних чисел на отрезке $[l, r]$ **чётной** длины (пример: $[1, 2, 3, 4, 5, 6], l = 2, r = 5 \rightarrow [1, 3, 2, 5, 4, 6]$),
 - `get(i)` — вывод числа на позиции i .

$\mathcal{O}(\log n)$ на запрос, online (n — текущая длина последовательности).
3. Придумайте структуру данных на основе AVL дерева, позволяющую online за $\mathcal{O}(\log n)$ отвечать на следующие запросы:
 - `insert(x, y)` ($0 \leq y < 5$)
 - `delete(x)`
 - `add(l, r, value)` — добавить по модулю 5 $value$ ко всем y , для которых $l \leq x \leq r$
 - `sum(l, r)` — сумма (обычная, не по модулю) всех y , для которых $l \leq x \leq r$
4. Пусть в обычное несбалансированное бинарное дерево поиска добавляются **различные** элементы в порядке $x_1, x_2, \dots, x_n$. Придумайте алгоритм, поддерживающий для каждой вершины её глубину и ссылки на родителя и детей и обновляющий эту информацию при добавлении элемента в дерево за $\mathcal{O}(\log n)$.

Пояснение: если делать добавление несбалансированном деревом, на запрос может уйти $\Omega(n)$ времени.
5. Придумайте структуру данных на основе AVL дерева, реализующую список с online операциями:
 - `make(x_0, x_1, x_2, \dots, x_{n-1})` — создать список со значениями. $\mathcal{O}(n)$.
 - `get(list, i)` — получить i -тый элемент списка длины n . $\mathcal{O}(\log n)$.
 - `reverse(list)` — развернуть список. $\mathcal{O}(1)$.
 - `splitAt(list, k)` — разделить список длины n на префикс длины k и оставшийся суффикс. $\mathcal{O}(\log n)$.
 - `concat(list1, list2)` — сконкатенировать два списка суммарной длины n . $\mathcal{O}(\log n)$.

Дополнительные задачи

6. Напишите на каком-нибудь строго типизированном функциональном языке с зависимыми типами (например, на **Agda**) реализацию исправления произвольного дисбаланса и добавления в **AVL** дерево так, чтобы типизацией гарантировался:
- (a) **AVL**-инвариант.
 - (b) **AVL**-инвариант и порядок на ключах.
7. Постройте тест, на котором «недо-**AVL**-дерево», которое делает только малые вращения, делает $\omega(n \log n)$ операций после n запросов.
- Формально: изначально дерево пусто, нужно n раз вызвать `add(root,  $x_i$ )` для некоторой последовательности x_i , что суммарное время работы $\omega(n \log n)$.
- Либо докажите, что такого теста нет.

1.2 Решения

1. Построим AVL-дерево, где для каждой вершины: левое поддерево имеет высоту h , правое поддерево имеет высоту $h - 1$.

При удалении узла из правого поддерева самой глубокой части дерева баланс нарушается в $\log n$ вершинах на пути к корню, где каждое нарушение требует одного или двух вращений.

2. Используем декартово дерево по неявному ключу, в котором элементы упорядочены по положению в последовательности, а структура сбалансирована случайными приоритетами.

Каждая вершина хранит:

- значение элемента;
- размер поддерева;
- флаг ленивой операции `swap_pairs` (для отложенного применения `twiddle`).

Рассмотрим сами операции:

- `insert(i, x)`: разбиваем дерево на две части по позиции i , вставляем новый узел и сливаем три дерева.
- `get(i)`: спускаемся в дерево, сравнивая i с размерами левого поддерева. На каждом шаге вызываем `push`, чтобы применить ленивые флаги.
- `twiddle(l, r)`: разбиваем дерево на три части: префикс до l , отрезок $[l, r]$ и суффикс после r . На элементы средней части вешаем ленивые флаги `swap_pairs`, сливаем обратно.
- Операция `push` применяется в `get`, `insert` и других, и при наличии флага `swap_pairs`:
 - рекурсивно передаёт флаг детям;
 - переставляет пары значений в поддереве

3. ...

4. ...

5. ...

6. ...

7. ...